

# PHASE 5 – PROJECT DEVELOPMENT

## 5.1 Introduction

The development phase involves implementing the AI BlogNest backend according to the requirements and design specifications.

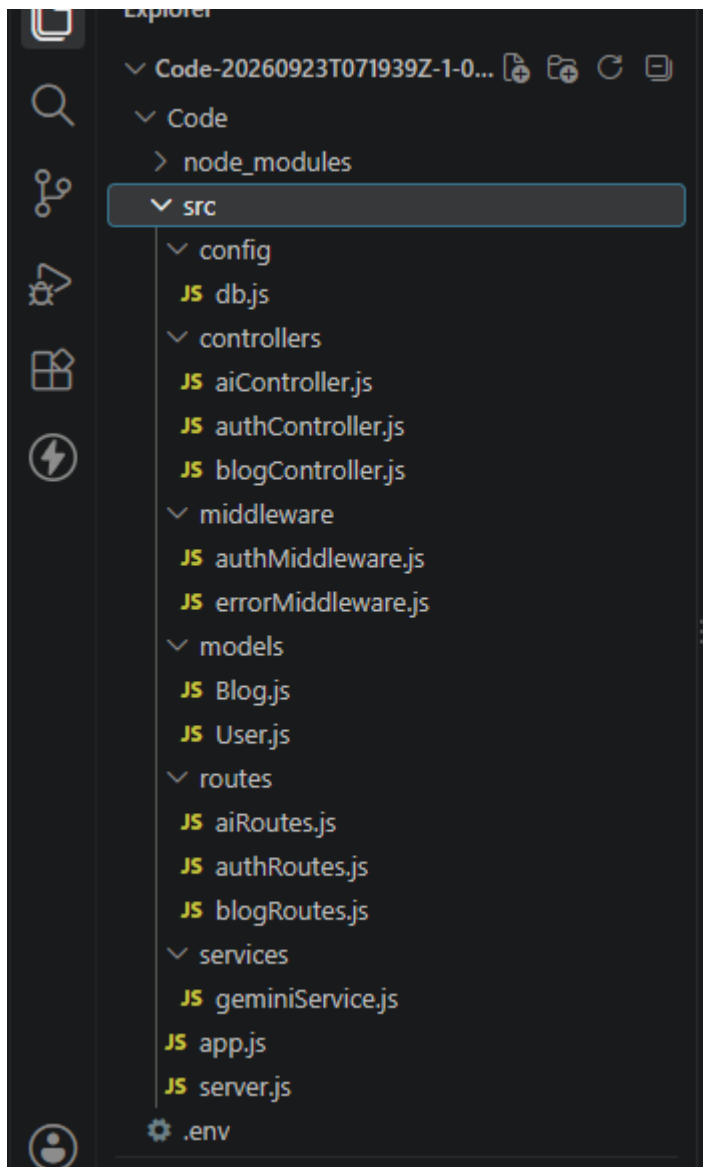
The application is developed using Node.js and Express.js, with MongoDB used for persistent data storage.

## 5.2 Project Structure

The backend follows a modular structure containing separate components for:

- Server configuration
- Routes
- Controllers
- Models
- Middleware
- Services
- Database configuration
- AI integration

This separation improves maintainability and makes individual modules easier to modify.



## 5.3 Backend Development

Node.js provides the runtime environment for executing the backend application.

Express.js is used to create REST API routes and handle HTTP requests.

The server processes requests, applies middleware, invokes controllers, communicates with the database, and returns JSON responses.

## 5.4 Authentication Development

The authentication system provides secure access to the application.

JWT tokens are used to maintain stateless authentication between the client and server.

Passwords are protected using bcrypt.js hashing before they are stored in the database.

## 5.5 Authorization

Role-based authorization controls access to protected resources.

The system supports:

- Admin
- Editor
- Author
- Reader

Middleware checks the authenticated user's role before allowing access to restricted operations.

## 5.6 Blog Development

The blog module provides CRUD operations.

CRUD represents:

- Create
- Read
- Update
- Delete

Users with appropriate permissions can create and manage blog posts.

Blog information can contain title, content, media information, likes, author reference, and timestamps.

## 5.7 AI Development

Google Gemini AI is integrated into the backend to provide intelligent content services.

The AI functionality can assist with:

- Blog content generation.
- Content summarization.
- FAQ assistance.
- WeatherWise insights.
- FitTrack recommendations.

The AI services receive suitable input from the application and return generated responses that can be processed by the backend.

## 5.8 Security Implementation

The application includes multiple security mechanisms:

- JWT authentication.
- Password hashing.
- Role-based authorization.
- Input validation.
- Input sanitization.
- Rate limiting.
- Centralized error handling. □ Structured logging.

These mechanisms help protect the backend against unauthorized access and common application-level vulnerabilities.